

Task 1: Clustering and Dimensionality Reduction

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

December 3, 2025

Contents

Context	2
1 Part A	3
2 Part B	7
3 Part C	11

Context

In this exercise, our aim is to analyze a high dimensional dataset using dimensionality reduction techniques and unsupervised learning. For this purpose, we will use the following congressional votes dataset that is provided with the assignment:

- The file "p1_congress_1984_votes.csv" contains a matrix $X \in \{-1, 0, 1\}^{435 \times 16}$ indicating the votes of 435 U.S. House of Representatives congress members on 16 key issues in the congress of 1984. Here, -1, 0 and 1 denote *reject*, *neutral* and *accept* votes respectively.
- The file "p1_congress_1984_party_affiliations.csv" contains a vector Y of size 435 x 1 indicating the party affiliations (Republican or Democrat) of 435 congress members in the congress of 1984. See [data source](#) for more information.

1 Part A

- On the congressional votes dataset given in "p1_congress_1984_votes.csv", apply principal component analysis (PCA). Plot the cumulative variance explained by top k principal components (with highest eigenvalues) as a function of k (see "example_pca_figure.png"). How many principal components do you think are enough to sufficiently summarize the data (according to the explained variance)?
- Next, project the data onto the first 3 principal components with highest eigenvalues. For each of the three principal component pairs (PC1-PC2, PC1-PC3, PC2-PC3), draw a scatter plot of congress members colored according to their party affiliations (as given in "p1_congress_1984_party_affiliations.csv"). Which of the principal component pair separates the congress members best according to their party affiliations? Are the congress members with the same party affiliation seem to be clustered according to their votes on the congress?

The PCA implementation and cumulative variance code are given below:

```
def congress_pca(data: np.ndarray, n_components: int = None) -> tuple:
    """
    Applies PCA to the congressional voting data.

    :param data: Feature matrix
    :type data: np.ndarray
    :param n_components: Number of components to keep
    :type n_components: int or None
    :returns: PCA model, transformed data, explained variance ratio
    :rtype: tuple
    """

    pca = PCA(n_components=n_components)
    data_pca = pca.fit_transform(data)
    explained_variance = pca.explained_variance_ratio_

    return pca, data_pca, explained_variance

def plot_cumulative_variance(explained_variance: np.ndarray,
                             output_filename: str):
    """
    Plots cumulative variance explained by top k principal components.

    :param explained_variance: Array of explained variance ratios
    :type explained_variance: np.ndarray
    :param output_filename: Filename for output figure
    :type output_filename: str
    """

    cumulative_variance = np.cumsum(explained_variance)
    k_values = np.arange(1, len(cumulative_variance) + 1)

    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    plt.plot(k_values, cumulative_variance, marker='o',
             linewidth=2, markersize=6, color='steelblue')

    plt.xlabel('Number of Principal Components (k)', fontsize=14, labelpad=15)
    plt.ylabel('Cumulative Variance Explained', fontsize=14, labelpad=15)
    plt.title('Cumulative Variance Explained by Top k Principal Components',
             fontsize=16, pad=20)
    plt.grid(True, alpha=0.3)
    plt.xlim(0, len(k_values) + 0.5)
```

```
plt.ylim(0, 1.05)

plt.tight_layout()
os.makedirs("../figures", exist_ok=True)
plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)
```

Figure 1: PCA and cumulative variance visualization implementation.

Figure 2 shows the cumulative variance explained over k principal components:

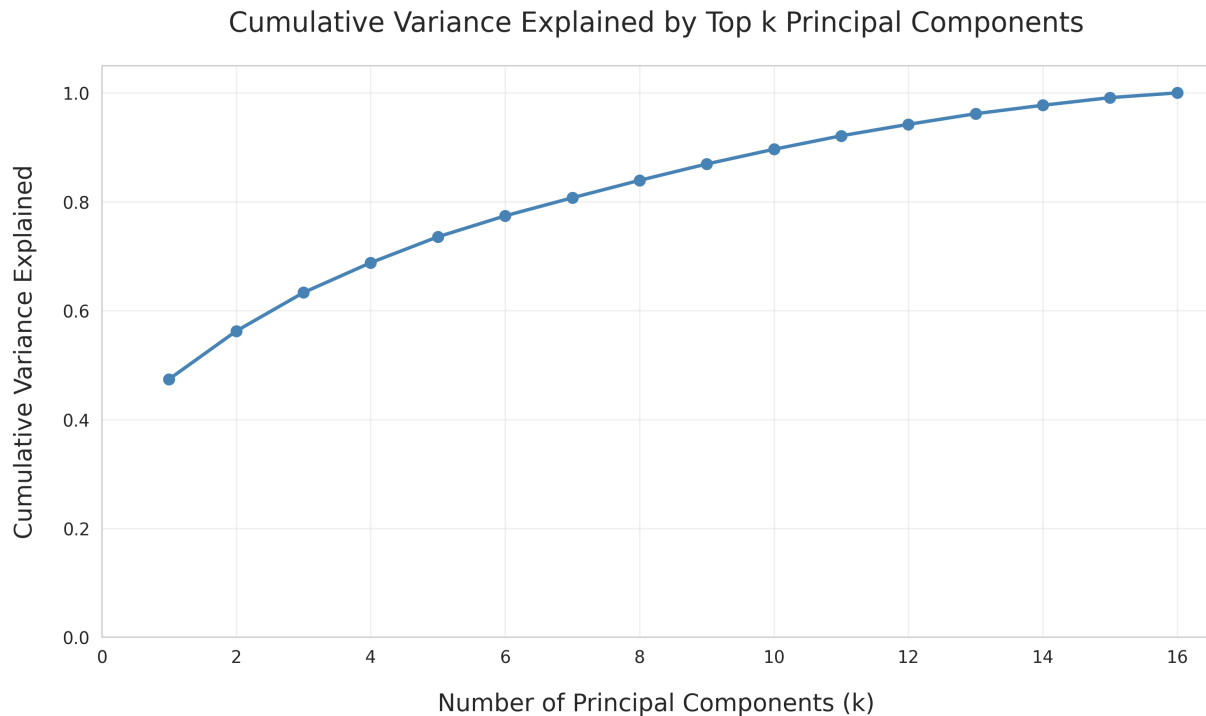


Figure 2: Cumulative variance explained by top k principal components.

PC1 alone covers 47.4% of the variance, and the first seven components together achieve 80% with lesser value in additional components past that point. I would argue this number of the PCs well-represents the voting data while still maintaining value from the dimensionality reduction, though adding more up to around 10-11 components before returns are severely diminished could be appropriate for prioritizing fidelity/separation in the voting patterns. The separation produced by the PC projections are shown below alongside their corresponding implementation:

```
def plot_pc_scatter(data_pca: np.ndarray, party_labels: np.ndarray,
                    pc_x: int, pc_y: int, output_filename: str):
    """
    Plots scatter plot of two principal components colored by party.

    :param data_pca: pca-transformed data
    :type data_pca: np.ndarray
    :param party_labels: Party affiliation labels
    :type party_labels: np.ndarray
    :param pc_x: Index of PC for x-axis
    :type pc_x: int
    :param pc_y: Index of PC for y-axis
    :type pc_y: int
    :param output_filename: Filename for output figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))
```

```

# Creates color mapping for parties
unique_parties = np.unique(party_labels)
colors = {'republican': 'red', 'democrat': 'blue'}

for party in unique_parties:
    mask = party_labels == party
    party_lower = party.lower()
    color = colors.get(party_lower, 'gray')

    plt.scatter(data_pca[mask, pc_x], data_pca[mask, pc_y],
                c=color, label=party.capitalize(),
                alpha=0.6, s=50, edgecolor='white', linewidth=0.5)

plt.xlabel(f'PC{pc_x + 1}', fontsize=14, labelpad=15)
plt.ylabel(f'PC{pc_y + 1}', fontsize=14, labelpad=15)
plt.title(f'Congress Voting Separation on PC{pc_x + 1} vs PC{pc_y + 1}',
          fontsize=16, pad=20)
plt.legend(fontsize=11, loc='best')
plt.grid(True, alpha=0.3)

plt.tight_layout()
os.makedirs("../figures", exist_ok=True)
plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 3: PC projections scatter plot function.

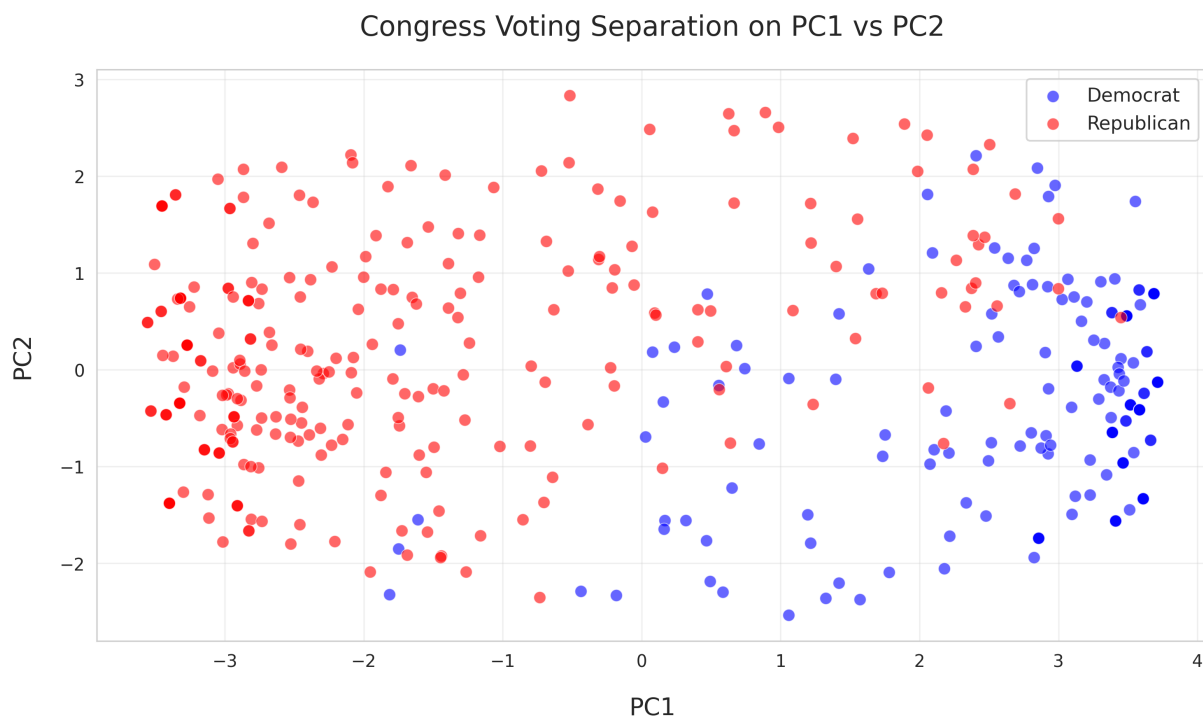


Figure 4: Congress members projected onto PC1 vs PC2, colored by party affiliation.

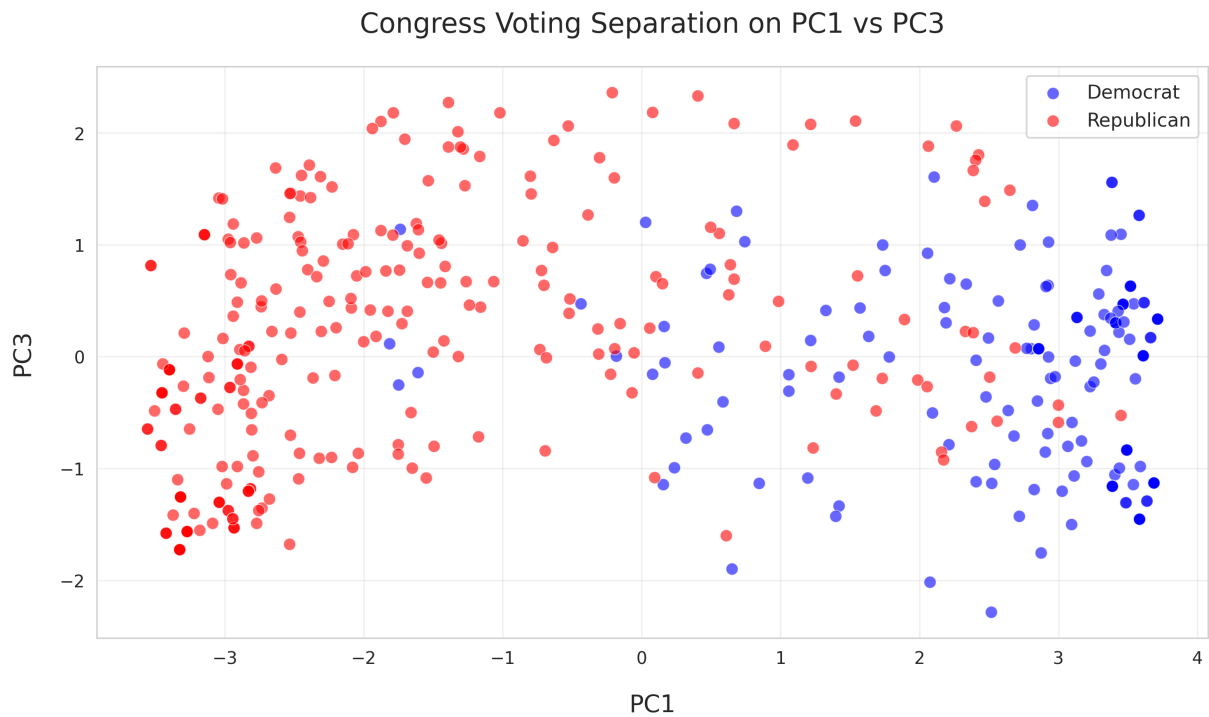


Figure 5: Congress members projected onto PC1 vs PC3, colored by party affiliation.

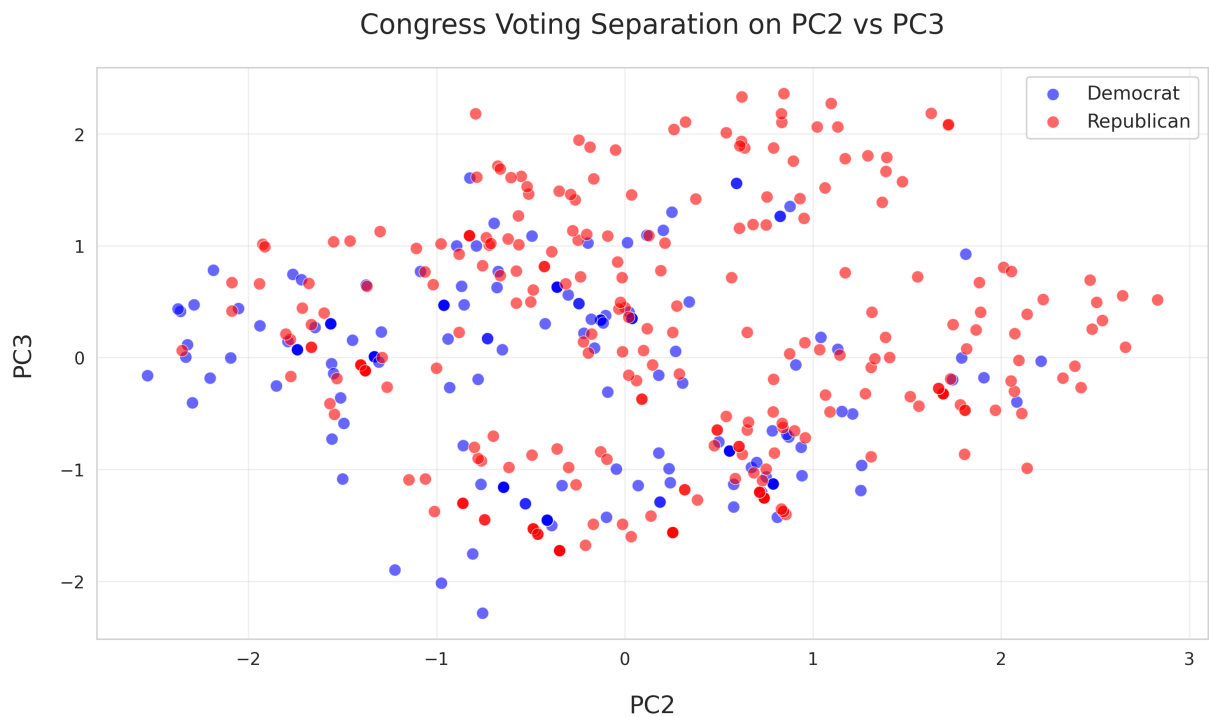


Figure 6: Congress members projected onto PC2 vs PC3, colored by party affiliation.

The PC1-PC2 projection produced the best separation of the congress members by political party. PC1-PC3 performs similar, but it is visually apparent that PC2 does marginally better to pull (R) members in the positive direction and (D) in the negative; PC2-PC3 shows much of the value lost from PC1, which as we saw largely dominates the cumulative variance. From the PC2-PC3 plot, we also get a better look at the better separation achieved by PC2 over PC3 more directly. As you would expect, the separation is immediately visually apparent along PC1, with (R) members on the left and (D) on the right with an expected small number of moderate players floating in the middle and just a handful leaning toward the opposing party.

The congress members of the same political party do appear to cluster together in this plot based on their voting patterns, suggesting that party affiliation is a decent indicator of voting behavior on key congressional issues, with PC1 being a fairly suitable party line dimension, as it covers nearly half of all the variance.

2 Part B

- Use your favorite clustering algorithm to cluster the congress members into two groups based on their congress votes on 16 issues. Make sure to explain the clustering algorithm and the distance function that you use to cluster the congress members. Visualize these groups with scatter plots on the first two top principal components you identified in part (a). Are these groups seem visually separated? How much do they agree with the party affiliations?
- Assess the statistical significance of the clustering you found using permutation tests. For this purpose, define a score to measure the quality of the clustering (for example, this could be the objective function of the K-means algorithm). Compute that score on the clustering you found on the original dataset. Now, obtain a permuted dataset by permuting each congress member's votes randomly across different matters (this will make the votes random and independent of each other, but will preserve the distribution of Reject/Neutral/Accept for each individual member). Then cluster this permuted dataset using the same algorithm you used to cluster the original dataset. Compute the score of the clustering again. Repeat this randomization process a large number of times (as allowed by computation resources). Now compare the distribution of the clustering scores you obtained on the permuted instances to the score you obtained on the original dataset. Based on this comparison, can you conclude that the original dataset is significantly clustered? Explain why.

I applied K-means clustering with two clusters to partition the congress members based on their voting patterns across the 16 issues. K-means minimizes the within-cluster sum of squared distances (inertia) using Euclidean distance, given as $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$. The algorithm then iteratively assigns each congress member to the nearest cluster centroid to it and updates the centroids position to the average position of the congress members assigned to it in the current iteration until their positions achieve a sufficient level of convergence i.e. the position updates of the centroids stop exceeding some satisfactory threshold. The implementation and the visualization of the clusters are shown below, which I included alongside the original projection from part A:

```
def cluster_kmeans(data: np.ndarray, n_clusters: int = 2,
                  random_state: int = 42) -> tuple:
    """
    Applies K-means clustering to the data.

    :param data: Feature matrix
    :type data: np.ndarray
    :param n_clusters: Number of clusters
    :type n_clusters: int
    :param random_state: Random seed
    :type random_state: int
    :returns: KMeans model, cluster labels, inertia score
    :rtype: tuple
    """
    kmeans = KMeans(n_clusters=n_clusters, random_state=random_state, n_init=10)
    cluster_labels = kmeans.fit_predict(data)
    inertia = kmeans.inertia_

    return kmeans, cluster_labels, inertia
```

Figure 7: K-means clustering implementation.

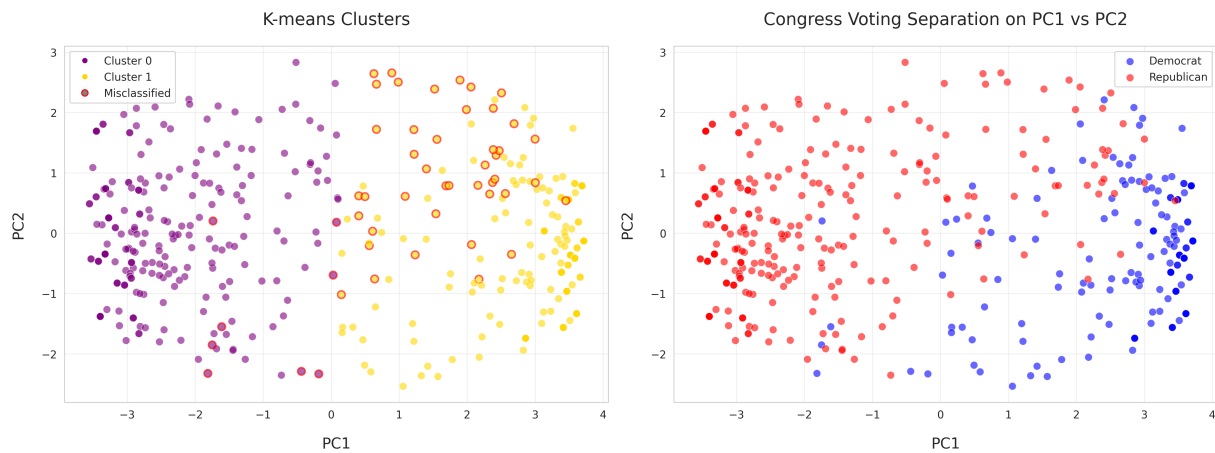


Figure 8: K-means clusters compared to party affiliations on PC1 vs PC2.

It is immediately apparent from the visual that the clusters are very well separated, very much along the PC1 origin boundary it appears. Regarding the party affiliations, kmeans is fairly weak to those members whose votings patterns stray from their parties, as it is the likely case that the centroids kept identifying these individuals as members of their cluster throughout the convergence due to their leaning behaviors. For that reason, I tried to capture the misclassification in the kmeans cluster overlay using red outlines, representing anyone regardless of party whose assigned cluster does not align with their political party. However, the voting patterns as they relate to party affiliation are still a rather good indicator, and so those whose voting behavior is as you would expected by party were largely assigned to the correct cluster, so I would still say the groups agree with their respective parties quite well.

To assess whether this clustering structure is statistically significant, I did exactly as the example in the instruction suggested and performed the permutation test using the objective function of the algorithm to measure the clustering quality. The observed result is 3778.34, and the implementation and visualization below show the score for 1000 permutations and their clusterings:

```
def permute_votes(votes: np.ndarray) -> np.ndarray:
    """
    Permutes votes randomly across issues for each congress member.

    :param votes: Original votes matrix
    :type votes: np.ndarray
    :returns: Permuted votes matrix
    :rtype: np.ndarray
    """
    permuted = votes.copy()
    for i in range(votes.shape[0]):
        permuted[i, :] = np.random.permutation(votes[i, :])

    return permuted

def permutation_test_clustering(votes: np.ndarray, n_permutations: int = 1000,
                               n_clusters: int = 2) -> tuple:
    """
    Performs permutation test for clustering significance.

    :param votes: Original votes matrix
    :type votes: np.ndarray
    :param n_permutations: Number of permutations
    :type n_permutations: int
    :param n_clusters: Number of clusters
    :type n_clusters: int
    :returns: Original inertia, null distribution, p-value
    :rtype: tuple
    """
```

```

# Cluster original data
_, _, original_inertia = cluster_kmeans(votes, n_clusters=n_clusters)

# Generates null distribution
null_inertias = np.zeros(n_permutations)
for i in range(n_permutations):
    permuted_votes = permute_votes(votes)
    _, _, null_inertias[i] = cluster_kmeans(permuted_votes,
                                           n_clusters=n_clusters,
                                           random_state=i)

p_value = (np.sum(null_inertias <= original_inertia) + 1) / (n_permutations + 1)

return original_inertia, null_inertias, p_value

def plot_permutation_results(original_inertia: float, null_inertias: np.ndarray,
                             p_value: float, output_filename: str):
    """
    Plots permutation test results.

    :param original_inertia: Inertia from original data
    :type original_inertia: float
    :param null_inertias: Null distribution of inertias
    :type null_inertias: np.ndarray
    :param p_value: Computed p-value
    :type p_value: float
    :param output_filename: Filename for output figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    plt.figure(figsize=(10, 6))

    plt.hist(null_inertias, bins=30, color='steelblue',
             alpha=0.7, edgecolor='white')

    plt.text(0.95, 0.95, f'Observed: {original_inertia:.2f} (p = {p_value:.3f})',
             transform=plt.gca().transAxes, verticalalignment='top',
             horizontalalignment='right', fontsize=12,
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    plt.xlabel('K-means Inertia', fontsize=14, labelpad=15)
    plt.ylabel('Frequency', fontsize=14, labelpad=15)
    plt.title('Distribution of K-Means Inertia for 1000 Permutations', fontsize=16, pad=20)
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    os.makedirs("../figures", exist_ok=True)
    plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 9: Vote permutation functions and visualization function for null distribution.

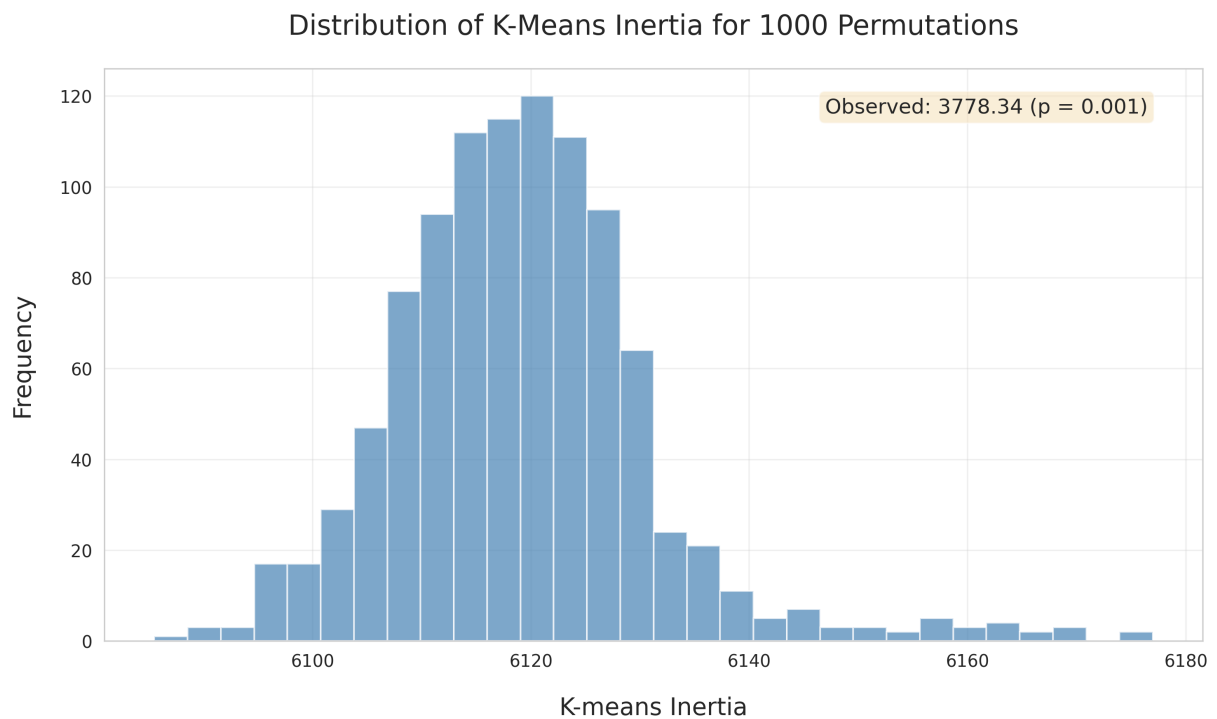


Figure 10: Null distribution of K-means inertia from permutation test.

The observed statistic is far lower than any value in the null distribution as expected from the discussion of the members whose voting patterns don't align with the affiliation, so for that reason I chose to exclude a visual marker of the observed statistic and left it as an annotation. Including it was ruining the scaling of the objective function axis and obfuscating the visual.

That being said, the observed statistic is far lower than the overwhelming majority of the distribution mass; where the average case measures at around 6120, the observed test statistic is only 3778.34. Along with its p-value of 0.001, the test result suggests that obtaining an observed clustering this strong is extremely unlikely under the null hypothesis, and we can reject. This indicates that the original data is significantly clustered.

3 Part C

- Now quantify the agreement of the clusters with the party affiliations (for example, using the mutual information between cluster membership and party affiliation).
- Repeat the clustering analysis using the first two principal components (instead of all 16 votes). Again, quantify the agreement of the clusters with the party affiliations. Which clustering (using principal components vs. using all 16 votes) agrees with the party affiliations more? Comment on why this might be the case.

To quantify the agreement of the clusters with the party affiliations, I used mutual information as the example suggests. The visualization code is shown below for the comparison visual of the different clusterings:

```
def plot_comparison(votes_pca: np.ndarray, cluster_labels_full: np.ndarray,
                   cluster_labels_pca: np.ndarray, output_filename: str):
    """
    Plots side-by-side comparison of clustering on full data vs PCs.

    :param votes_pca: pca-transformed data
    :type votes_pca: np.ndarray
    :param cluster_labels_full: Cluster assignments from full data
    :type cluster_labels_full: np.ndarray
    :param cluster_labels_pca: Cluster assignments from pca data
    :type cluster_labels_pca: np.ndarray
    :param party_labels: Party affiliation labels
    :type party_labels: np.ndarray
    :param output_filename: Filename for output figure
    :type output_filename: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")
    _, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

    cluster_colors = ['purple', 'gold']

    cluster_0_full_mean_pc1 = np.mean(votes_pca[cluster_labels_full == 0, 0])
    cluster_1_full_mean_pc1 = np.mean(votes_pca[cluster_labels_full == 1, 0])
    flip_full = cluster_0_full_mean_pc1 > cluster_1_full_mean_pc1

    cluster_0_pca_mean_pc1 = np.mean(votes_pca[cluster_labels_pca == 0, 0])
    cluster_1_pca_mean_pc1 = np.mean(votes_pca[cluster_labels_pca == 1, 0])
    flip_pca = cluster_0_pca_mean_pc1 > cluster_1_pca_mean_pc1

    # Clusters on full data
    for cluster in range(2):
        mask = cluster_labels_full == cluster
        color_idx = 1 - cluster if flip_full else cluster
        ax1.scatter(votes_pca[mask, 0], votes_pca[mask, 1],
                   c=cluster_colors[color_idx], label=f'Cluster {cluster}',
                   alpha=0.6, s=50, edgecolor='white', linewidth=0.5)

    ax1.set_xlabel('PC1', fontsize=14, labelpad=15)
    ax1.set_ylabel('PC2', fontsize=14, labelpad=15)
    ax1.set_title('K-means on Full Data', fontsize=16, pad=20)
    ax1.legend(fontsize=10, loc='best')
    ax1.grid(True, alpha=0.3)

    # Clusters on pc1-pc2
    for cluster in range(2):
        mask = cluster_labels_pca == cluster
```

```

color_idx = 1 - cluster if flip_pca else cluster
ax2.scatter(votes_pca[mask, 0], votes_pca[mask, 1],
            c=cluster_colors[color_idx], label=f'Cluster {cluster}',
            alpha=0.6, s=50, edgecolor='white', linewidth=0.5)

ax2.set_xlabel('PC1', fontsize=14, labelpad=15)
ax2.set_ylabel('PC2', fontsize=14, labelpad=15)
ax2.set_title('K-means on PC1-PC2', fontsize=16, pad=20)
ax2.legend(fontsize=10, loc='best')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
os.makedirs("../figures", exist_ok=True)
plt.savefig(f"../figures/{output_filename}", bbox_inches='tight', dpi=300)

```

Figure 11: Comparison visualization function for full data vs PCA clustering.

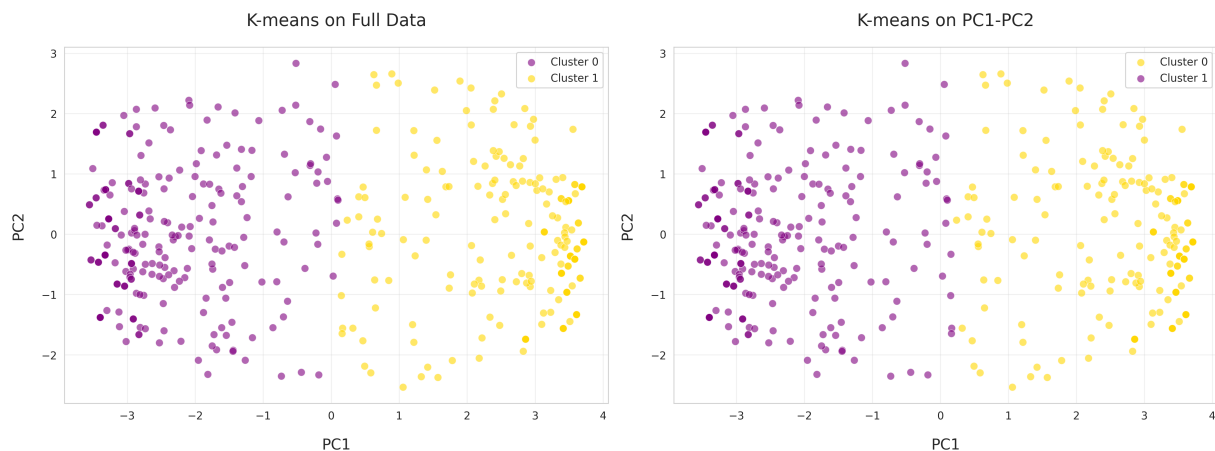


Figure 13: Side-by-side comparison of K-means clustering on 16 votes versus PC1-PC2 only.

The mutual information I obtained for the voting data and just the two PCs were 0.5097 and 0.4851 respectively. Both clusterings show decent agreement between the cluster assignments and the party affiliations, however the full data clustering shows just slightly better performance, as we would expect. I intended to show with the comparison visual the very slight differences in classification between the two approaches, which are readily apparent from the vertical party line down the PC1 origin boundary.

Regarding the result, PC1 and PC2 together explain 56.24% of the variance in the voting data, whereas the rest of the information about voting patterns lies in all of the components that were left out. In excluding them, the clustering is based only on these two dominant directions of variance, so it makes sense that the clustering would be more inexact and less-representative of the subtler patterns of agreement and disagreement across all of the issues that may align certain individuals better with their actual party membership. It is nonetheless impressive though how similar of a clustering could be achieved just from the first two PCs, where on visual inspection, the two approaches only differed in classification of 4 members of Congress.